BACHELOR OF COMPUTER SCIENCE

Department of Computer Science & Engineering

Final Year Project Submission — Academic Year 2025–2026

PROJECT SYNOPSIS

CHATIFY

A Real-Time Full-Stack Chat Application with Voice & Video Calling

A modern communication platform supporting instant messaging, group chats, audio/video calls, and live presence — built on React, Node.js, Firebase, and Supabase.

Submitted By

Nitish Kumar

Final Year — Bachelor of Computer Science

Roll No. / Enrolment No. : _____

Guided By

_____

(Guide / Supervisor Name)

Academic Year: 2025 – 2026

DECLARATION

I, **Nitish Kumar**, a student of the Final Year Bachelor of Computer Science programme, hereby declare that the project titled **"Chatify — A Real-Time Full-Stack Chat Application with Voice & Video Calling"** submitted in partial fulfilment of the requirements for the degree of Bachelor of Computer Science is an original piece of work carried out by me under the guidance of my project supervisor.

The information presented in this synopsis is true and correct to the best of my knowledge and belief. No part of this work has been submitted to any other university or institution for any degree or diploma.

Nitish Kumar	**Guide / Supervisor**	**HOD**
(Student)	(Signature & Stamp)	(Department Seal)

ABSTRACT

📖 **Chatify** is a full-stack, production-ready real-time communication platform developed as a final-year engineering project. The application addresses the growing demand for seamless digital communication by combining instant text messaging, multimedia sharing, group collaboration, and peer-to-peer audio/video calling in one unified interface.

The system is built using a modern three-tier architecture: a **React 19** single-page application on the client, an **Express.js** REST API on the server, and **Supabase (PostgreSQL)** as the primary database with built-in real-time capabilities. Authentication is delegated to **Firebase Authentication** via Google OAuth, ensuring industry-standard identity management without storing raw credentials.

Real-time messaging is achieved through Supabase's WebSocket-based table subscriptions, while voice and video calls leverage the **WebRTC** protocol with Supabase acting as the signalling channel — eliminating the need for a dedicated Socket.io server. The application supports user presence (online/offline/last-seen), group administration, message deletion (for-self and for-everyone), media uploads, and a 24-hour status-story feature.

The entire stack is deployed on **Vercel**, with the database hosted on Supabase's managed cloud, offering auto-scaling and zero-downtime deployments. Row-Level Security (RLS) policies on all tables ensure that users can only access data they are authorised to see.

TABLE OF CONTENTS

1. INTRODUCTION

In the digital age, real-time communication has become a cornerstone of both personal and professional life. Applications such as WhatsApp, Telegram, and Discord have demonstrated the immense value of low-latency, feature-rich chat platforms. Building such a system from scratch, however, requires mastery of distributed systems, WebSocket protocols, database design, and cloud deployment — skills that sit at the frontier of modern software engineering.

Chatify was conceived as a final-year engineering project to demonstrate these competencies in a single, cohesive application. The name "Chatify" reflects its core purpose: to *chat-ify* — to bring people together through instant, reliable communication. The project synthesises academic knowledge from subjects including:

- **Web Technologies** — React component model, SPA routing, REST API design
- **Database Management** — Relational schema design, SQL, Row-Level Security, real-time subscriptions
- **Computer Networks** — WebRTC peer-to-peer protocol, ICE/STUN negotiation, signalling
- **Information Security** — OAuth 2.0, JWT authentication, data access policies
- **Cloud Computing** — Serverless deployment, managed database services, CDN delivery
- **Software Engineering** — MVC architecture, RESTful principles, modular design patterns

> **Project Repository:** `https://github.com/thenitishmind/chatify`
> **Live Demo:** Deployed on Vercel (Frontend + Backend) with Supabase (Database)
> **Development Period:** Academic Year 2025 – 2026

2. PROBLEM STATEMENT & OBJECTIVES

2.1 Problem Statement

Existing open-source chat demos are often either too simple (lacking real-time delivery, call support, or group management) or too opaque (closed-source commercial products). There is a gap for a fully documented, academically grounded, end-to-end chat application that a student can study, extend, and deploy independently.

2.2 Objectives

- Design and implement a real-time messaging system capable of instant delivery without polling.
- Implement secure, standards-based authentication using Google OAuth and JWT tokens.
- Build WebRTC-based peer-to-peer audio and video calling with a signalling mechanism.
- Support group conversations with role-based administration (admin / member).
- Deploy the entire system to a public cloud with auto-scaling and zero infrastructure management.
- Enforce data privacy using Row-Level Security policies at the database layer.
- Provide a clean, responsive user interface usable on both desktop and mobile browsers.

3. TECHNOLOGY STACK

3.1 Frontend

Technology	Version	Purpose
React	19.x	UI component library — SPA rendering
Vite	6.x	Build tool & dev server (HMR on port 5173)
React Router	v7	Client-side navigation & protected routes
Axios	1.x	HTTP client with Firebase JWT interceptor
Firebase JS SDK	11.x	Google OAuth sign-in, ID token retrieval
Supabase JS	2.x	Real-time table subscriptions (WebSocket)
GSAP	3.x	Smooth UI animations & transitions

Technology	Version	Purpose
CSS3	—	Responsive layout, dark-mode-ready styling

3.2 Backend

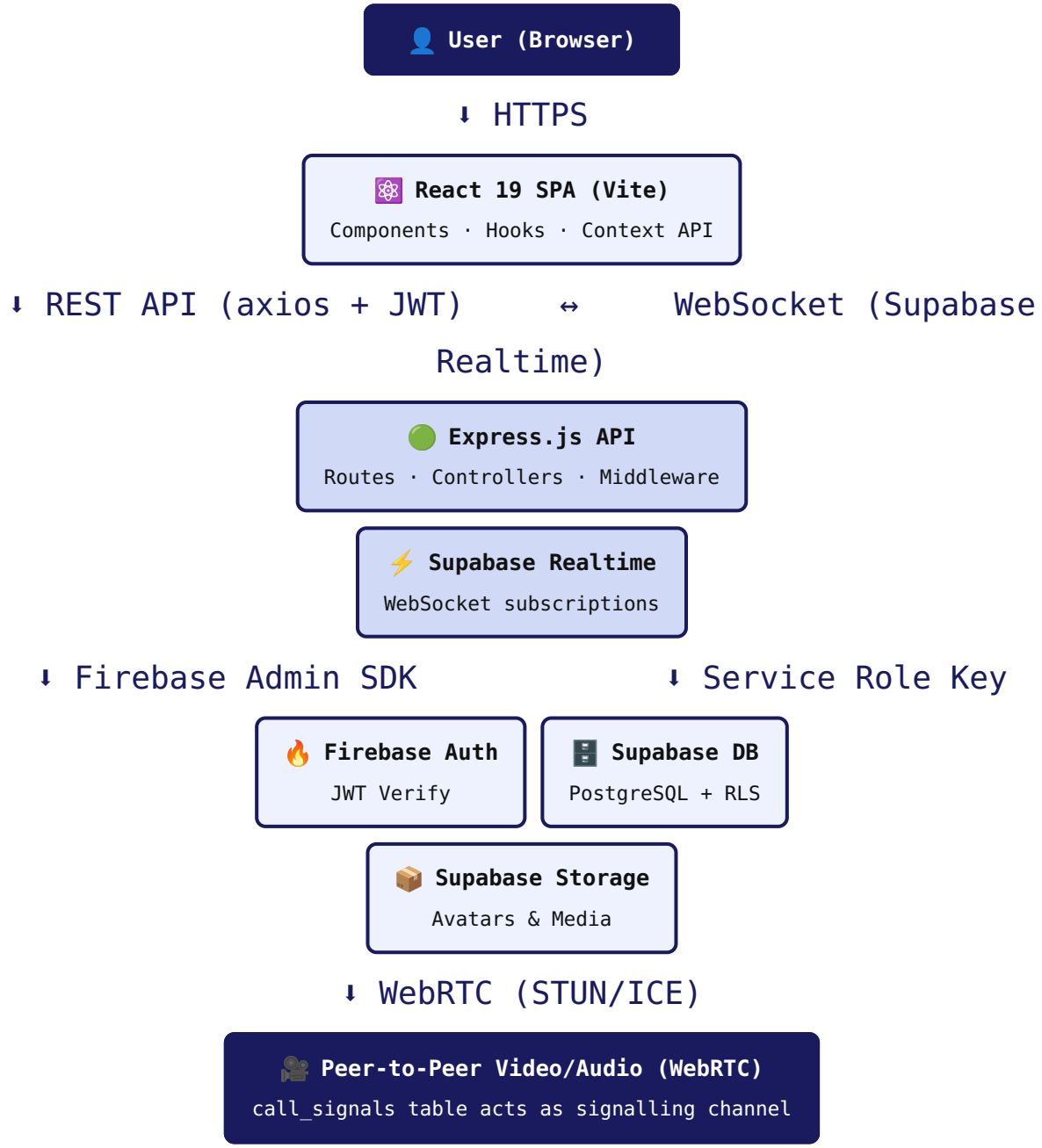
Technology	Version	Purpose
Node.js	18+	JavaScript runtime
Express.js	4.x	REST API framework (port 5000)
Firebase Admin SDK	12.x	Server-side JWT verification
Supabase JS	2.x	Database queries via service-role key (bypasses RLS)
Multer	1.x	Multipart file upload (avatars, media)
CORS	2.x	Cross-origin request management
Dotenv	16.x	Environment variable management

3.3 Cloud Services

Service	Role
Supabase	PostgreSQL database, real-time engine, object storage
Firebase Authentication	Google OAuth 2.0 identity provider
Vercel	Serverless deployment for both frontend (SPA) and backend (Node)

4. SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram



4.2 Authentication Flow

1. User clicks *Sign in with Google* on the React frontend.
2. Firebase JS SDK opens the Google OAuth consent screen and returns a `Firebase ID Token`.
3. Frontend sends `POST /api/auth/verify` with the ID token in the `Authorization` header.
4. Express `auth` middleware decodes and verifies the JWT using **Firebase Admin SDK**.
5. Backend upserts a row in the `profiles` table in Supabase and returns the profile.

6. If it is a new user (`isNewUser: true`), the frontend redirects to `/profile-setup` .
7. Every subsequent API request automatically attaches the latest Firebase ID token via an **Axios request interceptor**.

4.3 Real-Time Message Delivery Flow

1. User types a message and hits Send → frontend calls `POST /api/messages/:conversationId` .
2. Express controller validates membership, saves the message to Supabase, and updates `last_message` .
3. Both sender and recipient have active Supabase real-time subscriptions on the `messages` table filtered by `conversation_id` .
4. Supabase pushes a `INSERT` event over WebSocket; the `useMessages` hook appends the new message instantly — **no polling required**.

4.4 WebRTC Call Signalling Flow

1. Caller initiates call → SDP *offer* inserted into `call_signals` table.
2. Callee's Supabase subscription fires → `IncomingCallModal` appears.
3. Callee accepts → SDP *answer* inserted into `call_signals` .
4. Both peers exchange *ICE candidates* via `call_signals` inserts.
5. WebRTC PeerConnection completes — direct media stream established (bypasses server).
6. On end, an *end* signal is inserted and both peers close the connection.
7. Backend logs the call to `call_history` and posts a system message in the conversation.

5. DATABASE DESIGN

The database is hosted on **Supabase (PostgreSQL)**. All tables have *Row-Level Security (RLS)* enabled, meaning the database itself enforces access control — a user can only read and write rows they are authorised to access. The backend uses a *service-role key* (which bypasses RLS) so controllers can perform administrative operations. The frontend uses the *anon key* only for real-time subscriptions.

5.1 Entity-Relationship Overview

profiles		
id	uuid	PK · FK → auth.users
email	text	
phone	text	
display_name	text	
username	text UNIQUE	
avatar_url	text	
bio	text	
status	text	
is_online	boolean	
last_seen	timestamptz	

conversations		
id	uuid	PK
is_group	boolean	
participants	uuid[]	
group_name	text	
group_avatar	text	
group_admin	uuid	
last_message	text	
last_message_at	timestamptz	
created_at	timestamptz	

messages		
id	uuid	PK
conversation_id	uuid	FK → conversations
sender_id	uuid	FK → profiles
content	text	
message_type	text	
media_url	text	
media_type	text	
is_read	boolean	
deleted_for	uuid[]	

call_signals		
id	uuid	PK
caller_id	uuid	
callee_id	uuid	
call_type	text (audio/video)	
signal_type	text (offer/answer/ice/ end)	
signal_data	jsonb	
created_at	timestamptz	

call_history		
id	uuid	PK
caller_id	uuid	
callee_id	uuid	
call_type	text	
status	text (completed/missed/rejected)	
duration	integer (seconds)	
conversation_id	uuid	

group_members		
id	uuid	PK
group_id	uuid	FK → conversations
user_id	uuid	FK → profiles
role	text (admin/member)	
joined_at	timestamptz	

5.2 Real-Time Tables

The following tables have Supabase Realtime replication enabled:

- `messages` — instant delivery without polling
- `conversations` — sidebar updates when new conversation is created
- `call_signals` — WebRTC signalling channel (replaces Socket.io)

6. FEATURES & MODULES

🔐

Authentication Module

Google OAuth via Firebase. ID tokens verified server-side. New users complete a profile-setup wizard (username, avatar, bio).

💬

Real-Time Messaging

WebSocket-powered instant delivery. Text, image, and video messages. Delete-for-self and delete-for-everyone. Message context menu.

👥

Group Conversations

Create groups with a name and avatar. Role-based access (admin / member). Group admin can manage membership.

🎥

Audio & Video Calls

WebRTC peer-to-peer. No media relayed through server. Mute, camera toggle, end-call controls. Incoming call modal with accept/reject.

📞

Call History

Every call logged to `call_history`. Status: completed, missed, or rejected. Duration in seconds. System message in chat.

🟢

Online Presence

Tracks `is_online` boolean and `last_seen` timestamp per user. Updated on login and logout/ tab-close.

🔍

User Search

Search by username or display name. Instantly start a 1-on-1 conversation from search results.

🖼️

Media Uploads

Avatars stored in Supabase `avatars/` bucket. Chat media in `media/` bucket. Multer handles multipart upload on backend.

📋

Status Stories

24-hour ephemeral status updates. Stored in `status_stories` table with automatic expiry tracking.

📱

Responsive Design

CSS3 flexbox/grid layout. Works on desktop and mobile browsers. Separate style sheets for auth, chat, and call views.

6.1 Frontend Module Breakdown

Module	Key Files	Responsibility
Auth	LoginPage.jsx, ProfileSetup.jsx	Google sign-in, profile wizard
Chat Layout	ChatLayout.jsx, Sidebar.jsx	Two-panel layout, conversation list
Chat Window	ChatWindow.jsx, MessageBubble.jsx	Message display, scrolling, timestamps
Input	MessageInput.jsx	Text input, file picker, send
Groups	GroupCreate.jsx	Group name, avatar, member selection
Calls	CallScreen.jsx, CallControls.jsx, IncomingCallModal.jsx	WebRTC UI, controls, modal
Profile	ProfileSettings.jsx	Edit display name, avatar, bio, status

7. KEY CODE WALKTHROUGH

7.1 Express Server Entry Point (`backend/src/server.js`)

```
import express from 'express';
import cors from 'cors';
import dotenv from 'dotenv';
import authRoutes         from './routes/auth.js';
import userRoutes         from './routes/users.js';
import messageRoutes      from './routes/messages.js';
import conversationRoutes from './routes/conversations.js';
import statusRoutes       from './routes/status.js';
import callRoutes         from './routes/calls.js';
import { errorHandler }   from './middleware/errorHandler.js';

dotenv.config();
const app = express();

app.use(cors({ origin: process.env.FRONTEND_URL, credentials: true }));
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

app.use('/api/auth',          authRoutes);
app.use('/api/users',         userRoutes);
app.use('/api/messages',      messageRoutes);
app.use('/api/conversations',  conversationRoutes);
app.use('/api/status',        statusRoutes);
app.use('/api/calls',         callRoutes);
app.use(errorHandler);

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

7.2 Authentication Middleware (`backend/src/middleware/auth.js`)

```
import { auth } from '../config/firebase-admin.js';

export const verifyToken = async (req, res, next) => {
  const header = req.headers.authorization;
  if (!header?.startsWith('Bearer ')) return res.status(401).json({ error:
'No token' });

  try {
    const token   = header.split('Bearer ')[1];
    const decoded = await auth.verifyIdToken(token);      // Firebase Admin
SDK
    req.user = { uid: decoded.uid, email: decoded.email, phone:
decoded.phone_number };
    next();
  } catch (err) {
```

```
    res.status(401).json({ error: 'Invalid token' });
  }
};
```

7.3 Axios Interceptor — Auto-Attach JWT (`frontend/src/services/api.js`)

```
import axios from 'axios';
import { auth } from './firebase.js';

const api = axios.create({ baseURL: import.meta.env.VITE_API_URL || '/
api' });

api.interceptors.request.use(async (config) => {
  const user = auth.currentUser;
  if (user) {
    const token = await user.getIdToken();        // always fresh Firebase
JWT
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export default api;
```

7.4 Real-Time Message Subscription (`frontend/src/hooks/useMessages.js`)

```
import { useEffect, useState } from 'react';
import { supabase } from '../services/supabase.js';
import api from '../services/api.js';

export function useMessages(conversationId) {
  const [messages, setMessages] = useState([]);

  useEffect(() => {
    if (!conversationId) return;

    // Initial fetch
    api.get(`/messages/${conversationId}`).then(r => setMessages(r.data));

    // Real-time subscription
    const channel = supabase
      .channel(`messages:${conversationId}`)
      .on('postgres_changes', {
        event: 'INSERT',
        schema: 'public',
        table: 'messages',
        filter: `conversation_id=eq.${conversationId}`,
      }, payload => {
        setMessages(prev => [...prev, payload.new]);
      })
      .subscribe();

    return () => supabase.removeChannel(channel);   // cleanup on unmount
  }, [conversationId]);
```

```
  return messages;
}
```

7.5 Message Controller (`backend/src/controllers/messageController.js` — excerpt)

```
export const sendMessage = async (req, res) => {
  const { conversationId } = req.params;
  const { content, message_type = 'text', media_url, media_type } = req.body;
  const senderId = req.user.uid;

  // Verify sender is a participant
  const { data: convo } = await supabase
    .from('conversations')
    .select('participants')
    .eq('id', conversationId)
    .single();

  if (!convo?.participants?.includes(senderId))
    return res.status(403).json({ error: 'Not a participant' });

  // Insert message
  const { data: message } = await supabase
    .from('messages')
    .insert({ conversation_id: conversationId, sender_id: senderId,
              content, message_type, media_url, media_type })
    .select().single();

  // Update conversation's last message
  await supabase.from('conversations').update({
    last_message: content || `[${media_type}]`,
    last_message_at: new Date().toISOString(),
  }).eq('id', conversationId);

  res.status(201).json(message);
};
```

7.6 WebRTC Call Context (`frontend/src/context/CallContext.jsx` — excerpt)

```
const startCall = async (targetUser, callType) => {
  const stream = await navigator.mediaDevices.getUserMedia({
    audio: true, video: callType === 'video',
  });
  localStreamRef.current = stream;

  const pc = new RTCPeerConnection({ iceServers: [{ urls:
'stun:stun.l.google.com:19302' }] });
  peerConnectionRef.current = pc;
  stream.getTracks().forEach(track => pc.addTrack(track, stream));

  // Send ICE candidates via Supabase table (signalling)
  pc.onicecandidate = async ({ candidate }) => {
    if (candidate) {
      await supabase.from('call_signals').insert({
```

```
        caller_id: currentUser.uid, callee_id: targetUser.id,
        call_type: callType, signal_type: 'ice-candidate',
        signal_data: { candidate },
      });
    }
  };

  // Create and send SDP offer
  const offer = await pc.createOffer();
  await pc.setLocalDescription(offer);
  await supabase.from('call_signals').insert({
    caller_id: currentUser.uid, callee_id: targetUser.id,
    call_type: callType, signal_type: 'offer',
    signal_data: { offer },
  });
};
```

7.7 AuthContext — Global Auth State (`frontend/src/context/AuthContext.jsx` — excerpt)

```
export function AuthProvider({ children }) {
  const [user, setUser]       = useState(null);
  const [profile, setProfile] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    return onAuthStateChanged(auth, async (firebaseUser) => {
      if (firebaseUser) {
        const res = await api.post('/auth/verify');
        setProfile(res.data.profile);
        setUser(firebaseUser);
      } else {
        setUser(null); setProfile(null);
      }
      setLoading(false);
    });
  }, []);

  const logout = async () => {
    await api.post('/auth/logout');   // mark offline in DB
    await signOut(auth);
    setUser(null); setProfile(null);
  };

  return (
    <AuthContext.Provider value={{ user, profile, loading, logout }}>
      {children}
    </AuthContext.Provider>
  );
}
```

8. API ENDPOINTS

All endpoints are protected by the `verifyToken` middleware (except health-check). Requests must carry an `Authorization: Bearer <firebase-id-token>` header.

8.1 Authentication `/api/auth`

Method	Path	Description
POST	`/verify`	Verify Firebase JWT, upsert profile in Supabase, return user + isNewUser flag
PUT	`/profile`	Update display name, username, bio, status
POST	`/avatar`	Upload avatar image (multipart/form-data) → Supabase Storage
GET	`/check-username/:username`	Check if username is already taken
POST	`/logout`	Set `is_online=false` and `last_seen=now()`

8.2 Users `/api/users`

Method	Path	Description
GET	`/search?q=query`	Search profiles by username or display_name (case-insensitive ILIKE)
GET	`/me`	Return current user's profile
GET	`/:id`	Return any user's public profile

8.3 Conversations `/api/conversations`

Method	Path	Description
GET	`/`	Fetch all conversations for the current user (direct + groups)
POST	`/`	Create or retrieve existing 1-on-1 conversation with another user
POST	`/group`	Create a new group conversation with initial member list

8.4 Messages `/api/messages`

Method	Path	Description
GET	`/:conversationId`	Fetch last 50 messages (paginated), filtered by deleted_for
POST	`/:conversationId`	Send a new message (text or media)
DELETE	`/:messageId/for-me`	Soft-delete: add current user to `deleted_for` array
DELETE	`/:messageId/for-everyone`	Hard soft-delete: set `deleted_for_everyone=true` (sender only)

8.5 Calls `/api/calls`

Method	Path	Description
GET	`/`	Fetch call history for current user
POST	`/end`	Log completed call: status, duration, conversation_id; post system message

8.6 Status `/api/status`

Method	Path	Description
GET	`/`	Get user's status stories
PUT	`/`	Post or update a 24-hour status update

9. DEPLOYMENT & DEVOPS

9.1 Project Folder Structure

```
chatify/
├── backend/
│   ├── src/
│   │   ├── config/        (firebase-admin.js · supabase.js)
│   │   ├── controllers/   (auth · message · conversation · user · status ·
call)
│   │   ├── middleware/    (auth.js · errorHandler.js)
│   │   ├── routes/        (auth · users · messages · conversations · status
· calls)
│   │   └── server.js
│   ├── package.json
│   └── vercel.json        ← Vercel serverless Node config
│
├── frontend/
│   ├── src/
│   │   ├── context/       (AuthContext.jsx · CallContext.jsx)
│   │   ├── features/      (auth/ · chat/ · call/)
│   │   ├── hooks/         (useMessages.js · useConversations.js)
│   │   ├── services/      (api.js · firebase.js · supabase.js)
│   │   └── styles/
│   ├── index.html
│   ├── vite.config.js
│   └── vercel.json        ← SPA rewrite (/*)
│
└── database/
    ├── migration.sql      ← Schema + RLS policies
    └── migration_v2.sql   ← Schema updates
```

9.2 Environment Variables

Side	Variable	Purpose
Backend	`FIREBASE_PROJECT_ID`	Firebase project identifier
	`FIREBASE_PRIVATE_KEY`	Service account private key
	`FIREBASE_CLIENT_EMAIL`	Service account email
	`SUPABASE_URL`	Supabase project URL
	`SUPABASE_SERVICE_ROLE_KEY`	Bypass RLS for admin ops
Frontend	`VITE_FIREBASE_API_KEY`	Firebase web API key

Side	Variable	Purpose
	`VITE_FIREBASE_AUTH_DOMAIN`	OAuth redirect domain
	`VITE_SUPABASE_URL`	Supabase project URL
	`VITE_SUPABASE_ANON_KEY`	Realtime-only public key
	`VITE_API_URL`	Backend base URL

10. TESTING & SECURITY

10.1 Security Measures

- **JWT Verification:** Every API call verified server-side by Firebase Admin SDK — tokens cannot be forged.
- **Row-Level Security:** Database enforces access control independently of application code. A bug in the backend cannot expose other users' data.
- **Service Role Key Isolation:** Only the backend holds the service-role key; the frontend only has the anon key (read-only real-time).
- **CORS Policy:** Backend only accepts requests from the configured `FRONTEND_URL`.
- **Input Validation:** Membership checks before every message operation; sender-only enforcement for delete-for-everyone.
- **No Raw Credentials:** Passwords are never stored — authentication fully delegated to Google/Firebase.

10.2 Manual Testing Scenarios

#	Scenario	Expected Result
1	Google sign-in with new account	Redirect to /profile-setup
2	Send message in a 1-on-1 chat	Message appears in real-time for both users
3	Initiate a video call	Incoming modal appears; WebRTC stream established
4	Delete message for everyone	Message hidden for all participants
5	Create a group and add members	Group conversation appears in sidebar for all members
6	Close browser tab	is_online set to false, last_seen updated

11. FUTURE SCOPE

The following enhancements are identified for future development iterations:

- **End-to-End Encryption (E2EE):** Implement Signal Protocol or libsodium for message-level encryption so even the server cannot read content.
- **Push Notifications:** Firebase Cloud Messaging (FCM) to deliver messages when the app is closed.
- **Multi-Party Video Calls:** Integrate a media server (e.g., LiveKit or Daily.co) to support group video with more than two participants.
- **Message Reactions:** Emoji reactions with per-user tracking and real-time updates.
- **Message Search:** Full-text search across conversation history using PostgreSQL's `tsvector`.
- **Read Receipts:** Per-message delivery and read indicators (single tick, double tick, blue tick).
- **Disappearing Messages:** Time-bound messages that auto-delete after a configured period.
- **Mobile App:** React Native port using the same backend API for native iOS and Android experiences.
- **Admin Dashboard:** Analytics panel showing active users, message volume, and call statistics.
- **AI Chat Assistant:** Integrate Claude API to provide an in-chat AI assistant within any conversation.

12. CONCLUSION

Chatify demonstrates the design and implementation of a full-stack real-time communication platform using industry-standard technologies. The project successfully integrates *React 19*, *Express.js*, *Firebase Authentication*, *Supabase PostgreSQL*, and *WebRTC* into a cohesive, production-deployable system.

The architecture reflects best practices in modern web engineering: separation of concerns across frontend, backend, and database layers; stateless JWT-based authentication; real-time data propagation via WebSocket subscriptions; peer-to-peer media streams to minimise server load; and database-level Row-Level Security as a defence-in-depth measure.

Through this project, key computer-science concepts have been applied in a practical setting — from network protocol implementation (WebRTC/ICE) to distributed database design (PostgreSQL with real-time replication) and cloud-native deployment (Vercel serverless functions). The result is a scalable, secure, and feature-rich chat application that serves as a strong foundation for further research and commercial development.

13. REFERENCES

1. React Documentation — *react.dev*
2. Supabase Documentation — *supabase.com/docs*
3. Firebase Authentication Documentation — *firebase.google.com/docs/auth*
4. WebRTC API — MDN Web Docs — *developer.mozilla.org/en-US/docs/Web/API/WebRTC_API*
5. Express.js Guide — *expressjs.com/en/guide*
6. Vercel Deployment Documentation — *vercel.com/docs*
7. PostgreSQL Row-Level Security — *postgresql.org/docs/current/ddl-rowsecurity.html*
8. Tanenbaum, A.S. & Wetherall, D.J. — *Computer Networks*, 5th Ed., Pearson, 2011
9. Flanagan, D. — *JavaScript: The Definitive Guide*, 7th Ed., O'Reilly, 2020
10. Kleppmann, M. — *Designing Data-Intensive Applications*, O'Reilly, 2017

Submitted & Signed By

Student Name	Nitish Kumar
Programme	Bachelor of Computer Science (Final Year)
Academic Year	2025 – 2026
Project Title	Chatify — Real-Time Full-Stack Chat Application
Date	_____ / _____ / 2026

Nitish Kumar	**Project Guide**	**Head of Department**
(Student Signature)	(Signature & Date)	(Seal & Signature)